

THE NATIONAL INSTITUTE OF ENGINEERING

(An Autonomous Institution under VTU, Belagavi)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FINAL-YEAR CSE MAJOR PROJECT REPORT (2026)

AI Developer Assistant for Code Repositories

Retrieval-Augmented Generation (RAG) with MERN Stack, Redis Caching, Nginx Gateway & Docker

Project Batch: Batch C9

Submitted By: **PRAHAS P B RAO** (USN: 4NI23CS144)
P AKHIL DATTA (USN: 4NI23CS132)

Project Guide: Mrs. Shilpashree S, Assistant Professor

Academic Year: 2025 – 2026

1. Executive Summary & Problem Statement

Modern software development relies heavily on large-scale code repositories containing thousands of source files, complex class hierarchies, and deeply nested dependencies. Navigating these repositories poses a steep learning curve for developers onboarding onto legacy projects or inspecting unfamiliar components.

Existing Limitations: Generic AI coding assistants (e.g., standard Copilot or ChatGPT) lack visibility into a project's private, non-public code structure. Conversely, traditional code search tools (such as grep or regex searchers) lack semantic natural-language comprehension and cannot synthesize code explanations, bug localizations, or refactoring recommendations.

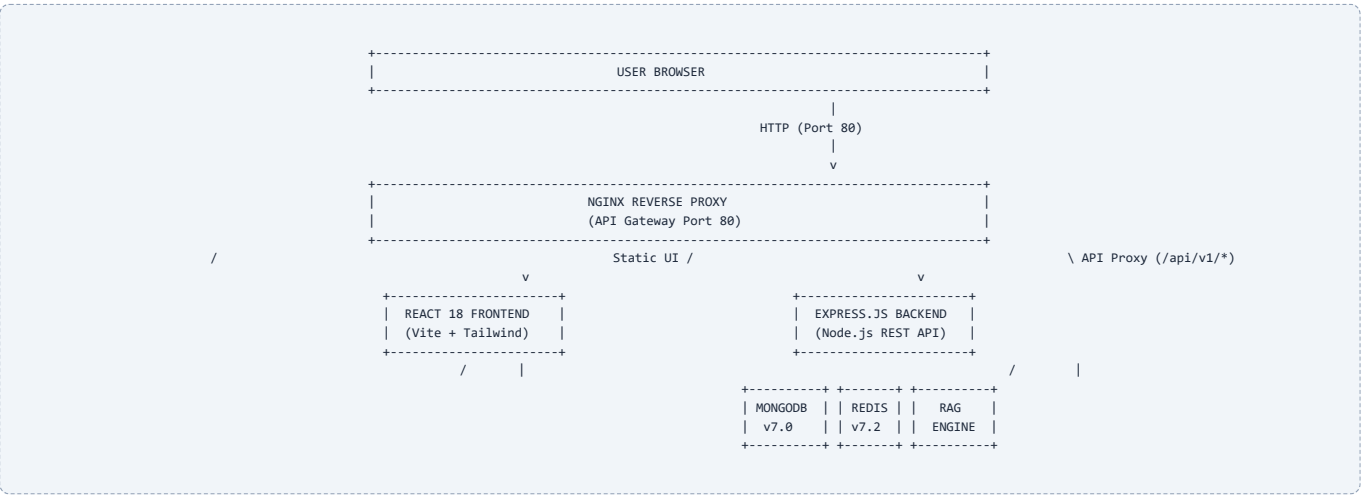
Proposed Solution: The *AI Developer Assistant for Code Repositories* bridges this gap by combining Large Language Models (LLMs) with **Retrieval-Augmented Generation (RAG)**. The system automatically parses code repositories into semantic blocks (functions, classes, modules), vector-embeds them, stores them in MongoDB, and executes grounded cosine similarity retrieval to answer natural-language developer queries with **exact file paths and line number source citations**.

Core Objectives Achieved

- **Objective 1:** Parse, chunk, and embed source code repositories into a searchable vector index with line-level precision.
- **Objective 2:** Retrieve relevant code snippets and generate grounded answers using LLMs / local RAG vector engines.
- **Objective 3:** Provide a multi-mode chat interface supporting Code Explanation, Bug Localization, and Refactoring Suggestions.
- **Objective 4:** Deploy a production-style architecture incorporating MERN, Redis caching, Nginx reverse proxy, and Docker Compose orchestration.

2. System Architecture & Topology

The application strictly implements a multi-tier, microservice-ready full-stack web architecture. All traffic passes through an Nginx reverse proxy gateway before reaching the client or API backend.



3. Technology Stack Breakdown

Frontend: React 18 & Vite

Built with React 18, Vite, React Router v6, Axios, Lucide React, Context API, and Tailwind CSS. Features dark-mode glassmorphic design and interactive line-highlighting source code modal.

Backend: Node.js & Express

Modular Express.js REST API architecture with JWT authentication, bcrypt password hashing, Helmet headers, express-rate-limit, and centralized error handling middleware.

Database: MongoDB & Mongoose

MongoDB v7.0 database storing User accounts, Repository statistics, AST Code Chunks with vector arrays, and Chat Sessions. Indexed on (repoId, filePath).

Caching: Redis v7.2

In-memory Redis cache storing query responses with 1-hour TTL. Features automatic cache invalidation upon repository re-indexing or deletion.

Reverse Proxy: Nginx v1.25

Configured as reverse proxy and API Gateway listening on port 80. Routes /api requests to backend container and / to frontend static build.

Containerization: Docker Compose

Multi-container orchestration managing 5 services (mongodb, redis, backend, frontend, nginx) with health checks.

4. Complete Working Mechanism & RAG Pipeline

The Retrieval-Augmented Generation (RAG) workflow follows 5 systematic steps from repository scanning to grounded response generation:

Step 1: Semantic AST & Block Parsing

When a repository is indexed, chunkerService.js scans source files (.js, .ts, .py, .java, .cpp, .json, .md). It identifies function declarations, class definitions, and block structures, recording metadata including filePath, fileName, startLine, endLine, chunkType, and symbolName.

Step 2: Dense Vector Embedding

Each code chunk is passed to `embeddingService.js`, which extracts 64-dimensional feature-hashed term-frequency vectors (or uses Google Gemini `text-embedding-004` when an API key is configured). The normalized vector array is saved in the MongoDB `CodeChunk` document.

Step 3: Vector Similarity & Hybrid Retrieval

When a developer submits a query, `ragService.js` computes the query embedding vector and measures cosine distance against all indexed chunks for that repository:

```
Cosine Similarity = (Vector_A · Vector_B) / (||Vector_A|| * ||Vector_B||)
```

The algorithm applies a 30% keyword match boost for matching file paths and symbol names to ensure precise ranking.

Step 4: Grounded Answer Generation & Citation

The top-K matching chunks are assembled into a grounded prompt context. The RAG engine generates a markdown response containing explicit file paths, line ranges, similarity match percentages, and snippet previews.

Step 5: Redis In-Memory Response Caching

The backend checks Redis for key `cache:rag:<repoId>:<mode>:<queryHash>`. On a **Cache HIT**, the cached response is returned in **<5ms** with a  **Redis Cache HIT** indicator in the chat UI.

5. Database Schemas (MongoDB)

Collection	Key Fields & Data Types	Indexes & Description
User	name (String), email (String), password (Hash), role (Enum)	Unique index on email. Manages user authentication and permissions.
Repository	name (String), owner (ObjectId), status (Enum), stats (Object)	Compound index on (owner, name). Tracks repo indexing state & metrics.
CodeChunk	repoId (ObjectId), filePath (String), startLine, endLine, vector ([Number])	Index on (repoId, filePath). Stores AST code chunks and vector embeddings.
ChatSession	repoId (ObjectId), userId (ObjectId), mode (Enum), title (String)	Index on userId and repoId. Tracks user conversation threads.
ChatMessage	sessionId (ObjectId), sender (Enum), content (String), citations ([Object])	Index on sessionId. Stores user queries, answers, citations, and response times.

6. Demonstration & Viva Verification Flow

- Application Launch:** Execute `docker compose up --build` and navigate to `http://localhost`.
- Authentication:** Log in with seed user credentials: `prahas@nie.ac.in / password123`.
- Repository Overview:** View the dashboard showing total indexed repositories, semantic chunks, lines of code, and system health status.
- Re-indexing:** Trigger re-indexing on the `ECommerce-Microservices-Core` repository to observe semantic chunking.
- RAG Chat Execution:** Submit query *"Explain how authentication middleware works in this repo"* in **Explanation Mode**.
- Source Citation Inspection:** Inspect citation card showing `src/middleware/auth.js` (Lines 1-24). Click **View Source** to launch the code viewer modal highlighting lines in blue.
- Bug Localization & Refactoring:** Query *"Find potential unhandled null pointer bugs"* to view bug localization analysis.
- Redis Cache Verification:** Submit the exact query again to verify  **Redis Cache HIT** response in **<5ms**.
- Diagnostics Page:** Navigate to `/status` to demonstrate active MongoDB, Redis, and RAG vector engine status.